Zen MoDE: Mixture of Distilled Experts Architecture

Technical Report v2025.06

Zen LM Research Team
research@zenlm.org

June 2025

Abstract

We present Zen MoDE (Mixture of Distilled Experts), the core architectural innovation underlying the Zen model family. Zen MoDE addresses two fundamental challenges in large language model scaling: computational efficiency via sparse expert routing, and knowledge transfer via distilled expert initialization. We introduce semantic routing, where experts specialize by input domain and linguistic style rather than by position-hashing or random assignment. Combined with distilled expert initialization from smaller dense teacher models, Zen MoDE achieves performance matching a 400B-parameter dense model using only 47B active parameters (480B total). We present the router architecture, expert specialization analysis, load balancing algorithm, and performance benchmarks comparing Zen MoDE against dense equivalents.

1 Introduction

Scaling language models requires navigating a fundamental tension: more parameters improve capability, but increase inference cost proportionally. Mixture-of-Experts (MoE) architectures offer an elegant resolution: maintain a large parameter count while activating only a sparse subset per forward pass.

Conventional MoE implementations route tokens to experts via a learned linear router, trained to maximize routing utility. In practice, this leads to load imbalance (popular experts become overloaded while others are underutilized) and poor expert specialization (experts do not develop interpretable specializations).

Zen MoDE introduces three innovations that address these limitations:

1. **Semantic routing**: Expert selection guided by semantic content embeddings, enabling interpretable domain and style specialization.
2. **Distilled expert initialization**: Each expert is initialized from a smaller dense teacher model specialized on a semantic domain, providing warm-start knowledge.
3. **Adaptive capacity factors**: Dynamic capacity allocation based on real-time expert load, preventing token dropping under skewed distributions.

2 Architecture

2.1 Overall Structure

Zen MoDE follows the standard Transformer architecture with MoE layers replacing every-other dense feed-forward network (FFN) layer:

- **Layer structure**: Alternating dense attention + dense FFN, MoE FFN pattern.

- **Expert count**: 128 experts per MoE layer.
- **Active experts**: Top-$K = 8$ experts per token ($K = 2$ for the 14B model).
- **Expert FFN size**: $4d_{\text{model}}$ hidden dimension (same as dense FFN equivalent).
- **Shared experts**: 4 experts always activated ("shared" experts, active for every token).

The total parameter count P_{total} and active parameters P_{active} for an L-layer model with N_E experts:

$$P_{\text{total}} = P_{\text{attn}} + P_{\text{shared}} + N_E \cdot P_{\text{expert}} \tag{1}$$
$$P_{\text{active}} = P_{\text{attn}} + P_{\text{shared}} + K \cdot P_{\text{expert}} \tag{2}$$

For the 480B Zen MoDE: $P_{\text{total}} = 480B$, $P_{\text{active}} = 47B$ (9.8% activation ratio).

2.2 Model Scale Configurations

Table 1: Zen MoDE model configurations

Model	Total Params	Active Params	Experts	K	Layers	d_{model}
Zen MoDE 14B	14B	2.4B	64	2	28	2048
Zen MoDE 72B	72B	8.1B	128	4	64	4096
Zen MoDE 236B	236B	22B	128	8	80	7168
Zen MoDE 480B	480B	47B	128	8	94	7680

3 Semantic Routing

3.1 Router Architecture

Conventional MoE routers apply a linear projection to the token representation and take the top-K softmax scores:

$$\text{router}_{\text{standard}}(x) = \text{TopK}(\text{softmax}(\mathbf{W}_r x), K) \tag{3}$$

The Zen MoDE semantic router adds a two-stage process:

Stage 1: Semantic Embedding. A lightweight embedding model f_s maps the input sequence prefix to a 256-dimensional semantic vector:

$$\mathbf{s} = f_s(\text{prefix}) \in \mathbb{R}^{256} \tag{4}$$

This semantic vector captures topic domain, linguistic register, and task type at the sequence level.

Stage 2: Expert Scoring. Token-level routing is biased toward experts whose learned prototype $\mathbf{p}_e$ is nearest to the semantic vector:

$$\text{score}(x, e) = \underbrace{\mathbf{w}_e^\top x}_{\text{token routing}} + \underbrace{\beta \cdot \text{sim}(\mathbf{s}, \mathbf{p}_e)}_{\text{semantic bias}} \tag{5}$$

where $\beta = 0.3$ balances local (token) and global (semantic) routing signals.

3.2 Expert Specialization

Table 2: Expert specialization analysis: top domains per expert cluster (128 experts, 16 clusters)

Cluster	Primary Domain	Secondary Domains
1–8	Code (Python, JS, Rust)	Algorithms, data structures
9–16	Mathematics (formal)	Proofs, theorems
17–24	Scientific writing	Academic, citations
25–32	Medical/clinical	Pharmacology, anatomy
33–40	Legal text	Contracts, statutes
41–48	Financial text	Earnings, risk, regulation
49–56	Creative writing	Fiction, poetry
57–64	Conversational	Dialogue, casual register
65–72	News and journalism	Current events, AP style
73–80	Technical documentation	Manuals, API docs
81–88	Multilingual (CJK)	Japanese, Chinese, Korean
89–96	Multilingual (Arabic)	Arabic, Farsi, Hebrew
97–104	Multilingual (Slavic)	Russian, Polish, Czech
105–112	Low-resource languages	African, SE Asian
113–120	Instruction following	Commands, structured output
121–128	Reasoning (chain-of-thought)	Logic, planning

Expert specialization is measured by the Jensen-Shannon divergence between the token distribution of expert e's input and the corpus-wide distribution. Higher divergence indicates stronger specialization. Zen MoDE experts achieve average JS-divergence of 0.42 versus 0.18 for standard top-K routing (2.3× improvement).

4 Distilled Expert Initialization

4.1 Teacher Model Preparation

For each semantic domain cluster c, we prepare a specialist dense teacher model:

1. Start from a 7B dense model pre-trained on the full corpus.
2. Fine-tune on domain-specific data (e.g., code corpus for code experts, PubMed for medical experts).
3. Extract the FFN weight matrices $\{W_1^c, W_2^c\}$ for each layer as expert initialization.

4.2 Expert Initialization Protocol

Expert e in cluster c is initialized by projecting the teacher FFN weights to expert dimensions:

$$W_{e,\text{init}} = \mathbf{P}_e \cdot W^c_{\text{teacher}} \cdot \mathbf{Q}_e \tag{6}$$

where $\mathbf{P}_e, \mathbf{Q}_e$ are random orthogonal projections that map from teacher dimensions to expert dimensions. This preserves the spectral properties of the teacher weights in a lower-dimensional subspace.

Table 3: Distilled initialization vs. random initialization (perplexity on held-out eval, lower is better)

Initialization	Step 1K	Step 10K	Step 100K
Random	48.2	18.4	8.4
Copy from dense base	22.4	12.8	7.8
Distilled (ours)	**16.4**	**10.2**	**7.2**

Distilled initialization provides significant early-training advantage, converging to final loss in 31% fewer training steps.

5 Load Balancing

5.1 Expert Load Imbalance Problem

Without load balancing, popular experts become overloaded while others are underutilized. Overloaded experts either drop tokens (capacity-limited) or become bottlenecks (unlimited capacity). Both outcomes degrade performance.

5.2 Auxiliary Load Balancing Loss

Zen MoDE minimizes an auxiliary load balancing loss alongside the main language modeling objective:

$$\mathcal{L}_{\text{balance}} = \alpha \cdot N \sum_{e=1}^{N_E} f_e \cdot P_e \tag{7}$$

where $f_e = \frac{1}{T} \sum_t \mathbf{1}[e \in \text{TopK}(t)]$ is the fraction of tokens routed to expert e, $P_e = \frac{1}{T} \sum_t p_t^e$ is the average routing probability for expert e, and $\alpha = 0.01$ is the load balancing coefficient.

5.3 Adaptive Capacity Factors

The capacity factor C limits the maximum tokens an expert can process per batch:

$$\text{capacity}_e = C \cdot \frac{\text{batch size} \cdot \text{seq len}}{N_E} \tag{8}$$

Zen MoDE uses adaptive capacity factors that adjust per-expert based on rolling load statistics:

$$C_e(t) = C_{\text{base}} + \gamma \cdot \left(\bar{f}_e - \frac{1}{N_E} \right) \tag{9}$$

Under-utilized experts ($\bar{f}_e < 1/N_E$) get reduced capacity, over-utilized experts get increased capacity, preventing token dropping under natural load distribution skew.

6 Benchmark Results

6.1 Performance vs. Dense Equivalent

Table 4: Zen MoDE vs. dense equivalent: MMLU 5-shot (%)

Model	Params	Active	MMLU
Dense 7B	7B	7B	63.4
Zen MoDE 14B	14B	2.4B	72.8
Dense 13B	13B	13B	67.2
Dense 70B	70B	70B	80.2
Zen MoDE 72B	72B	8.1B	84.8
Dense 400B (estimated)	400B	400B	90.4
Zen MoDE 480B	480B	47B	91.2

Zen MoDE 480B matches estimated dense-400B performance with only 47B active parameters—an 8.5× reduction in inference FLOPs.

6.2 Expert Utilization

Table 5: Expert utilization metrics (Zen MoDE 72B)

Metric	Standard Top-K	Zen MoDE
Utilization entropy	5.42 bits	6.81 bits (max 7 bits)
CV (coefficient of variation)	0.84	0.31
Token drop rate	3.2%	0.8%
Expert utilization below 50%	24 experts	8 experts
Expert utilization above 200%	18 experts	3 experts

6.3 Routing Quality

Table 6: Routing quality metrics

Metric	Standard Top-K	Semantic Routing
Domain consistency (same-domain token routing)	61.4%	84.2%
Expert JS-divergence	0.18	0.42
Router confidence (avg top score)	0.48	0.62
Cross-domain routing (noise)	38.6%	15.8%

7 Training Efficiency

7.1 Compute Budget

The MoDE architecture provides significant training compute savings over dense models at equivalent capacity:

$$\frac{\text{FLOPs}_{\text{MoDE}}}{\text{FLOPs}_{\text{dense-equivalent}}} = \frac{K + N_{\text{shared}}}{N_E} \approx \frac{12}{128} \approx 9.4\% \tag{10}$$

In practice, attention layers (non-sparse) and overhead reduce the realized savings to approximately 8× reduction in training FLOPs for the same total parameters.

7.2 Communication Overhead (Distributed Training)

Expert parallelism requires all-to-all communication to route tokens to the correct expert device. Communication overhead as a fraction of total training time:

Table 7: Expert parallelism communication overhead

Model	Expert Parallelism	Comm / Total	Effective Throughput
72B	8 nodes	8.4%	91.6% of compute
236B	16 nodes	11.2%	88.8% of compute
480B	32 nodes	14.8%	85.2% of compute

8 Conclusion

Zen MoDE demonstrates that principled mixture-of-experts design—semantic routing, distilled expert initialization, and adaptive load balancing—substantially improves over naive sparse routing approaches. The 480B Zen MoDE model matches estimated dense-400B performance at 8.5× lower inference FLOPs, and semantic routing increases expert specialization by 2.3× as measured by domain-routing consistency. Distilled expert initialization accelerates convergence by 31%, enabling efficient training of very large expert counts.

References

[1] Shazeer, N. et al. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. *ICLR*, 2017.

[2] Fedus, W. et al. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *JMLR*, 2022.

[3] Lepikhin, D. et al. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. *ICLR*, 2021.

[4] Jiang, A. et al. Mixtral of Experts. *arXiv:2401.04088*, 2024.

[5] Zuo, S. et al. Taming Sparsely Activated Transformer with Stochastic Experts. *ICLR*, 2022.